Unidad 7 · Implementación en OPL

Qué vas a poder hacer. Traducir un modelo algebraico a OPL separando el modelo (`.mod`) de los datos (`.dat`), correrlo, leer en su salida la solución, las holguras, los precios sombra y los costos reducidos, y encontrar un error a partir del mensaje de OPL o de una solución que no calza.

Lo que necesitas antes. Las unidades 4 y 5: escribir un modelo con conjuntos, índices, parámetros y variables, con sumatorias ($\sum$) y «para todo» ($\forall$). De la unidad 2, qué es una restricción activa. Del precio sombra basta la idea: cuánto mejora el objetivo con una unidad más de recurso (se formaliza en las unidades 10 y 12).

Tiempo. Unas 5 horas: 2 de lectura con el ejemplo abierto en el computador y 3 de práctica.

---

1. La idea: el modelo algebraico ya es casi el código

OPL (*Optimization Programming Language*) es el lenguaje de modelos de IBM ILOG CPLEX Optimization Studio. No se programa *cómo* resolver: se escribe el modelo casi igual que en el papel y CPLEX lo resuelve. Por eso el trabajo serio está antes, en el modelo en palabras y en notación algebraica.

Dos razones para no quedarse en la planilla:

- **Escala.** Una restricción $\forall r \in R$ es una sola línea, tenga R tres elementos o tres mil. En la planilla son tres mil filas, y Solver (el complemento de Excel) admite como máximo 200 variables y 100 restricciones.
- **Modelo y datos separados.** El mismo `.mod` sirve para la semana de invierno y la de verano: se cambia el `.dat` y nada más. Un error en una fórmula de planilla se copia a todas las celdas; en OPL el modelo se revisa una vez.

En el papel	En OPL
conjunto I de nombres	`{string} I = ...;`
conjunto $T = \{1, \ldots, 4\}$	`range T = 1..4;`
parámetro c_i	`float c[I] = ...;`
parámetro a_{ri}	`float a[R][I] = ...;`
variable $x_i \geq 0$	`dvar float+ x[I];`
variable entera o binaria	`dvar int+ n[I];` · `dvar boolean y[I];`
$\sum_{i \in I} c_i x_i$	`sum(i in I) c[i]*x[i]`
$\forall r \in R$	`forall(r in R)`
$\forall t \in T,\ t > 1$	`forall(t in T: t > 1)`
$\leq$, $\geq$, $=$	`<=`, `>=`, `==`
$\max$, $\min$	`maximize`, `minimize`

2. Un proyecto OPL

En CPLEX Studio todo vive en un **proyecto**: una carpeta con los archivos de un problema.

- `.mod` tiene el modelo. Un proyecto puede tener varios.
- `.dat` tiene los datos. Puede haber varios juegos de datos para el mismo modelo.
- `.ops` guarda opciones del solver (rara vez se toca).
- Una **configuración de ejecución** junta *un* `.mod` con los `.dat` que se le asignen. Lo que se ejecuta es la configuración, no el proyecto ni el archivo suelto: clic derecho sobre ella y «Ejecutar esta». Para llevarte el proyecto, copia o comprime la carpeta entera.

Fuera del IDE, el mismo modelo corre desde la terminal con `oplrun modelo.mod datos.dat`. Es lo que hace `herramientas/opl.py` de esta base, y así se corrió cada modelo de esta unidad.

Un `.mod` se lee de arriba abajo en este orden:

1. conjuntos y parámetros (lo que *se sabe*);
2. variables de decisión (lo que *se decide*);
3. expresiones de cálculo (lo que *se deriva* de las decisiones);
4. objetivo;
5. restricciones, dentro de `subject to { ... }`;
6. post-proceso: un bloque `execute { ... }` que corre después de resolver.

3. Conjuntos y parámetros

Conjuntos. `{string} Prendas = ...;` es un conjunto de nombres. `range Semanas = 1..4;` es un rango de enteros: sirve cuando el orden importa y hace falta escribir `t-1`. Con un conjunto de nombres no se puede restar 1; para eso existen `first(S)`, `last(S)`, `prev(S, e)`, `next(S, e)` y `ord(S, e)`.

Parámetros. `float cap[Recursos] = ...;` declara un dato indexado por un conjunto. Los tres puntos significan «este valor lo leo del `.dat`». Un parámetro se puede escribir en el `.mod` (`float K = 250;`), pero la buena práctica es dejar el `.mod` sin números.

El `.dat`. Los conjuntos van entre llaves y con comillas; los arreglos, entre corchetes y separados por espacios, **en el mismo orden del conjunto**; un arreglo de dos índices es una lista de filas:

```
Prendas  = {"poleras", "polerones"};
margen   = [1380 3940];
uso      = [ [30 90]
             [20 50] ];
```

`uso[r][i]` es la fila `r`, columna `i`: el primer índice declarado es la fila.

> **Cuidado.** Un arreglo con menos números que su conjunto no da error: OPL completa con ceros y resuelve igual. El ejercicio 7.9 muestra lo que eso hace con una mezcla.

Datos desde Excel (solo como referencia). En Windows, el `.dat` puede leer rangos de una planilla:

```
SheetConnection hoja("Datos.xlsx");
uso from SheetRead(hoja, "'Recursos'!C4:G6");
```

El archivo tiene que estar en la carpeta del proyecto y cerrado al ejecutar, y en Mac no funciona. Esta base no depende de Excel: todos sus `.dat` traen los números escritos.

---

4. Variables y expresiones

Declaración	Qué es	Cuándo
`dvar float+ x[I];`	continua, $x_i \geq 0$	casi siempre
`dvar float x[I];`	continua libre (puede ser negativa)	un cambio de inventario, una temperatura
`dvar int+ n[I];`	entera ≥ 0	buses, personas, lotes indivisibles
`dvar boolean y[I];`	0 o 1	encender una máquina, abrir una bodega
`dvar float+ x[I] in 0..100;`	con cotas	un tope físico de la variable

El `+` es la no negatividad. Sin él, OPL deja la variable libre. A veces no cambia nada; en un modelo de mínimo costo puede volverlo no acotado (sección 7).

`dexpr` calcula, no decide. `dexpr float usado[r in Recursos] = sum(i in Prendas) uso[r][i]*x[i];` no agrega variables: le pone nombre a una expresión. Sirve para escribir restricciones más legibles (`usado[r] <= cap[r]`) y para imprimir resultados («variables de estado», como se les dice en el taller). Si un `dexpr` no se usa en el objetivo ni en una restricción, igual se puede imprimir.

5. Objetivo y restricciones

```
maximize sum(i in Prendas) margen[i] * x[i];

subject to {
  forall(r in Recursos)
    Capacidad:
      usado[r] <= cap[r];
}
```

- Cada restricción termina en `;`. La igualdad se escribe `==`; un `=` solo es un error de sintaxis.
- **Ponle nombre.** `Capacidad:` delante de la restricción crea el arreglo de restricciones `Capacidad[r]`. Sin nombre no hay cómo pedirle su holgura ni su precio sombra.
- **Filtros.** `forall(i in Componentes: D[i] < 10000)` genera la restricción solo para los elementos que cumplen la condición. Igual con `sum(i in I: ...)`.
- **El índice tiene que existir.** Dentro de `sum(i in Prendas)` vale `i`; fuera, no. Sumar sobre el índice equivocado es el error de notación más frecuente del ramo.

6. `execute`: imprimir y leer la solución

El bloque `execute` va después de `subject to` y corre cuando CPLEX ya terminó. Su lenguaje es un JavaScript simplificado (IBM ILOG Script):

```
execute {
  writeln("margen = ", cplex.getObjValue());
  for (var r in Recursos)
    writeln("dual[", r, "] = ", Capacidad[r].dual);
}
```

Las propiedades que importan para leer la solución:

Propiedad	Qué dice	Signo que debes esperar
`x[i]`	el valor óptimo de la variable	
`Rest[r].slack`	cuánto falta para llegar al límite; 0 = restricción activa	siempre ≥ 0
`Rest[r].dual`	precio sombra: cuánto cambia el objetivo si el lado derecho sube en 1	máx. con $\leq$: ≥ 0 · mín. con $\geq$: ≥ 0 · mín. con $\leq$: ≤ 0
`x[i].reducedCost`	costo reducido: cuánto cambia el objetivo por cada unidad que se fuerza de una variable que está en 0	máx.: ≤ 0 · mín.: ≥ 0

El signo del dual es el de la derivada del objetivo. En un problema de **mínimo costo**, una restricción de capacidad ($\leq$) que aprieta tiene dual negativo: una unidad más de capacidad *baja* el costo. Leerlo como si fuera de máximo («vale −12, no sirve») es un error clásico.

Solo para modelos lineales continuos. Con una variable `int` o `boolean` el modelo es entero mixto; ahí `.dual` y `.reducedCost` responden `undefined` y CPLEX avisa `CPLEX Error 1017: Not available for mixed-integer problems.` (ejercicio 7.7).

Para guardar la salida en un archivo: `var f = new IloOplOutputFile("Solucion.txt");`, `f.writeln(...)` y `f.close();`.

7. Cuando OPL reclama: errores frecuentes y cómo leerlos

Los mensajes de abajo son reales: salen de correr los modelos de esta unidad con un error puesto a propósito (el verificador los vuelve a provocar). OPL indica la línea y la columna (`at 21:1-6`).

Mensaje de OPL	Qué pasó	Arreglo
`syntax error, unexpected dexpr, expecting ';'`	falta el `;` de la línea **anterior** (el número de línea apunta a la siguiente)	cierra la declaración anterior
`syntax error, unexpected =`	una restricción con `=`	`==`, `<=` o `>=`
`syntax error, unexpected ...`	un dato declarado como `dvar` (`dvar float cap[R] = ...;`)	las variables no se leen del `.dat`: es `float`
`Name "i" does not exist.`	un índice que ningún `sum` ni `forall` definió	usa el índice del `sum`
`Index out of bound for array "x": poleron.`	un nombre que no está en el conjunto (falta una letra)	copia el nombre exacto del `.dat`
`Index out of bound for array "uso": lavanda.`	índices al revés: `uso[p][r]` en un arreglo declarado `[Recursos][Plantas]`	respeta el orden de la declaración
`External data element "demanda" was not defined.`	el `.mod` pide un dato que el `.dat` no trae	agrégalo al `.dat` (o revisa la ortografía)
`Row 'Piso' infeasible, all entries at implied bounds.` y `<<< no solution`	modelo **infactible**: esa fila no se puede cumplir	revisa datos y sentido de la desigualdad
`... make problem unbounded.` y `<<< no solution`	modelo **no acotado** (en el ejemplo, un transporte sin el `+`)	no negatividad y restricciones que falten
`undefined` y `CPLEX Error 1017: Not available for mixed-integer problems.`	`.dual` en un modelo con `int` o `boolean`	no hay duales en un modelo entero

Dos errores **no** dan mensaje y son los peores: un arreglo corto en el `.dat` (se completa con ceros) y unidades mezcladas (capacidad en horas, consumo en segundos). Ambos dan una solución «óptima» de otro problema. La defensa es leer la salida con sentido común: ¿se usa lo que se tenía? ¿Los precios sombra están en una escala razonable?

8. Lo mismo en planilla y Solver

El taller 9 del ramo arma los mismos modelos en Excel. La equivalencia es directa:

Planilla con Solver	OPL
celdas de variables (las que cambia Solver)	`dvar`
celda objetivo con `SUMAPRODUCTO(margen; x)`	`maximize sum(i in I) margen[i]*x[i];`
lado izquierdo de cada restricción con `SUMAPRODUCTO`	`dexpr` o la suma dentro de la restricción
restricciones en bloque `F8:F9 <= G8:G9`	`forall(p in P) Nombre: usado[p] <= disp[p];`
«Convertir variables sin restricciones en no negativas»	`float+`
método «Simplex LP»	CPLEX, que reconoce solo que el modelo es lineal
informe de sensibilidad · Valor final (variables)	`x[i]`
informe de sensibilidad · Costo reducido	`x[i].reducedCost`
informe de sensibilidad · Precio sombra	`Nombre[p].dual`
Lado derecho menos Valor final (restricciones)	`Nombre[p].slack`

Los rangos «Aumento permisible» y «Disminución permisible» del informe son el tema de la unidad 12.

9. Ejemplo resuelto · La panadería del barrio

Una panadería de Chillán hornea por bandejas. Esta semana decide cuántas bandejas de **marraqueta**, **hallulla** y **pan amasado** producir.

	Marraqueta	Hallulla	Amasado	Disponible en la semana
Margen por bandeja	$6.000	$4.200	$6.500	
Horno (min/bandeja)	16	12	20	2.480 min
Amasadora (min/bandeja)	8	10	18	1.730 min
Harina (kg/bandeja)	5	4,5	6	885 kg
Se venden como máximo	80	125	60	

Paso 1 · En palabras. Se decide cuántas bandejas de cada pan hornear en la semana. Se busca el mayor margen. Limitan los minutos de horno y de amasadora, los kilos de harina y lo que se alcanza a vender de cada pan.

Paso 2 · Algebraico. Conjuntos: P panes, R recursos. Parámetros: m_p margen, a_{rp} uso del recurso r por bandeja de p, D_r disponible, V_p venta máxima. Variable: x_p bandejas de p.

$$\max \sum_{p \in P} m_p x_p \quad \text{s.a.} \quad \sum_{p \in P} a_{rp} x_p \le D_r \;\; \forall r \in R, \qquad x_p \le V_p \;\; \forall p \in P, \qquad x_p \ge 0 \;\; \forall p \in P$$

Paso 3 · El `.mod` (`modelos/u07_panaderia.mod`). Cada línea del algebraico tiene la suya:

```
/*********************************************
 * Unidad 7 · Ejemplo resuelto
 * Panadería de barrio: bandejas de pan por semana
 * Datos en u07_panaderia.dat. Base abierta de ingeniería.
 *********************************************/

{string} Panes    = ...;
{string} Recursos = ...;

float margen[Panes]          = ...;   // CLP por bandeja
float uso[Recursos][Panes]   = ...;   // minutos o kg por bandeja
float disp[Recursos]         = ...;   // minutos o kg por semana
float tope[Panes]            = ...;   // bandejas que se venden como máximo

dvar float+ x[Panes];                 // bandejas a hornear en la semana

dexpr float Margen = sum(p in Panes) margen[p] * x[p];
dexpr float usado[r in Recursos] = sum(p in Panes) uso[r][p] * x[p];

maximize Margen;

subject to {
  forall(r in Recursos)
    Recurso:
      usado[r] <= disp[r];
  forall(p in Panes)
    Venta:
      x[p] <= tope[p];
}

execute {
  writeln("margen = ", Margen);
  for (var p in Panes) {
    writeln("x[", p, "] = ", x[p]);
    writeln("costo_reducido[", p, "] = ", x[p].reducedCost);
    writeln("dual_venta[", p, "] = ", Venta[p].dual);
  }
  for (var r in Recursos) {
    writeln("holgura[", r, "] = ", Recurso[r].slack);
    writeln("dual[", r, "] = ", Recurso[r].dual);
  }
}
```

Paso 4 · El `.dat` (`modelos/u07_panaderia.dat`):

```
// Unidad 7 · ejemplo resuelto · datos de la panadería (una semana)

Panes    = {"marraqueta", "hallulla", "amasado"};
Recursos = {"horno", "amasadora", "harina"};

margen = [6000 4200 6500];

uso = [
  [16 12 20]        // horno (min/bandeja)
  [ 8 10 18]        // amasadora (min/bandeja)
  [ 5 4.5 6]        // harina (kg/bandeja)
];

disp = [2480 1730 885];
tope = [80 125 60];
```

En el `.dat` el decimal es un **punto** (`4.5`): OPL no entiende la coma.

Paso 5 · Correr y mirar la salida. `oplrun u07_panaderia.mod u07_panaderia.dat` imprime `OBJECTIVE: 900000` y, desde el `execute`:

```
margen = 900000
x[marraqueta] = 80
costo_reducido[marraqueta] = 0
dual_venta[marraqueta] = 400
x[hallulla] = 100
costo_reducido[hallulla] = 0
dual_venta[hallulla] = 0
x[amasado] = 0
costo_reducido[amasado] = -500
dual_venta[amasado] = 0
holgura[horno] = 0
dual[horno] = 350
holgura[amasadora] = 90
dual[amasadora] = 0
holgura[harina] = 35
dual[harina] = 0
```

Paso 6 · La misma salida, ordenada.

	Valor	Lectura
Plan	80 bandejas de marraqueta, 100 de hallulla, 0 de amasado	margen $900.000
Horno	holgura 0, dual 350	activa: cada minuto más de horno vale $350
Amasadora	holgura 90 minutos, dual 0	sobra: un minuto más no vale nada
Harina	holgura 35 kg, dual 0	sobra
Venta de marraqueta	dual 400	una bandeja más vendible sube el margen en $400
Pan amasado	costo reducido −500	cada bandeja forzada baja el margen en $500

Cómo se lee.

- **El horno manda.** Es la única restricción de recurso activa. Por eso el plan se ordena por margen *por minuto de horno*: marraqueta \$6.000/16 = \$375, hallulla \$4.200/12 = \$350 y amasado \$6.500/20 = \$325. La marraqueta llega a su tope, la hallulla usa el horno que queda y el amasado no entra.
- **El dual de la venta de marraqueta (\$400) no es su margen (\$6.000).** Una bandeja más de marraqueta usa 16 minutos de horno que se le quitan a la hallulla: $6.000 - 16 \cdot 350 = 400$.
- **El costo reducido del amasado (−500)** sale igual: sus 20 minutos de horno «cuestan» $20 \cdot 350 = 7.000$ pesos y deja solo \$6.500. Entraría al plan si su margen subiera \$500.
- **Decisión.** No compres harina ni pagues más turnos de amasadora. Si alguien arrienda horno por menos de \$350 el minuto (\$21.000 la hora), conviene, hasta el rango que calcula la unidad 12.

---

10. Errores típicos

Error	Cómo evitarlo
Confundir variable con parámetro: `dvar float cap[R] = ...;`	lo que se sabe es `float` (y viene del `.dat`); lo que se decide es `dvar`
Olvidar la no negatividad (`float` en vez de `float+`)	escribe `float+` por defecto; deja libre solo lo que de verdad puede ser negativo
Sumar sobre el índice equivocado (`sum(i in I) a[r][i]*x[r]`)	el índice de la variable tiene que ser el del `sum`
Índices al revés respecto de la declaración	lee la declaración: `uso[Recursos][Prendas]` se usa `uso[r][i]`
Leer el dual de un mínimo con el signo de un máximo	en mínimo, un $\leq$ que aprieta da dual negativo: más capacidad baja el costo
Pedir `.dual` en un modelo con `int` o `boolean`	los duales existen solo en el modelo lineal continuo
Arreglo corto en el `.dat` o unidades mezcladas	cuenta los números de cada arreglo y escribe la unidad al lado, en comentario
Dejar números dentro del `.mod`	el `.mod` lleva el modelo; el `.dat`, los datos

---

11. Práctica

Diez ejercicios: 7.1 a 7.4 básicos, 7.5 a 7.8 intermedios, 7.9 y 7.10 de desafío. Las pautas están en la sección 12 y los modelos resueltos en `modelos/u07_e*.mod`. Corre cada uno antes de mirar.

7.1 · Cervecería artesanal (básico, escribir). Una cervecería produce lager y stout por barril. Un barril de lager usa 3 h de maceración y 2 tanques-semana de fermentación; uno de stout, 5 h y 2 tanques-semana. Al mes hay 150 h de maceración y 80 tanques-semana. El margen es \$23.000 por barril de lager y \$29.000 por barril de stout. Escribe el `.mod` **sin** `.dat`, como en el primer taller: dos `dvar`, un `dexpr` con las horas de maceración usadas, las dos restricciones con nombre y un `execute` que imprima la solución y los dos precios sombra. Córrelo e informa el plan.

7.2 · Leer un modelo de mezcla (básico, leer). Este es `modelos/u07_e02_alimento.mod`, con su `.dat`:

```
{string} Ingredientes = ...;
{string} Nutrientes   = ...;

float costo[Ingredientes]              = ...;
float aporte[Nutrientes][Ingredientes] = ...;
float minimo[Nutrientes]               = ...;
float maximo[Nutrientes]               = ...;
float K                                = ...;

dvar float+ kg[Ingredientes];

minimize sum(i in Ingredientes) costo[i] * kg[i];

subject to {
  Total:
    sum(i in Ingredientes) kg[i] == K;
  forall(n in Nutrientes)
    Piso:
      sum(i in Ingredientes) aporte[n][i] * kg[i] >= minimo[n] * K;
  forall(n in Nutrientes: maximo[n] > 0)
    Techo:
      sum(i in Ingredientes) aporte[n][i] * kg[i] <= maximo[n] * K;
}
```

```
Ingredientes = {"pollo", "arroz", "maiz", "aceite"};
Nutrientes   = {"proteina", "grasa"};
costo  = [2380 870 640 1950];          // CLP por kg
aporte = [ [58 8 10   0]               // proteína (% del ingrediente)
           [ 9 1  5 100] ];            // grasa (%)
minimo = [24 10];                      // % mínimo en el lote
maximo = [0 16];                       // % máximo; 0 = sin techo
K = 250;                               // kg del lote
```

(a) Di en palabras qué decide, qué busca y qué lo limita. (b) Escríbelo en notación algebraica. (c) ¿Qué hace `: maximo[n] > 0` y cuántas restricciones `Techo` genera? (d) ¿Por qué los porcentajes se pueden dejar enteros sin dividir por 100? (e) Córrelo: ¿qué lleva el lote y cuánto cuesta? ¿Cuánto sube el costo si la proteína mínima pasa de 24 % a 25 %?

7.3 · Seis errores en un vivero (básico, corregir). Un vivero decide cuántas macetas de lavanda, romero y suculentas preparar en la semana (`modelos/u07_e03_vivero.dat` trae los datos). Este `.mod` tiene seis errores. Encuéntralos, di qué mensaje da OPL por cada uno (o por qué no da ninguno) y corrígelos. Con el modelo corregido, informa el plan.

```
{string} Plantas  = ...;
{string} Recursos = ...;

float margen[Plantas]          = ...;
float uso[Recursos][Plantas]   = ...;
dvar float cap[Recursos]       = ...;
float demanda[Plantas]         = ...;

dvar float x[Plantas]

maximize sum(p in Plantas) margen[p] * x[i];

subject to {
  forall(r in Recursos)
    Capacidad:
      sum(p in Plantas) uso[p][r] * x[p] <= cap[r];
  forall(p in Plantas)
    Demanda:
      x[p] = demanda[p];
}
```

7.4 · Leer la salida de una carpintería (básico, leer). Una carpintería decide cuántas mesas, sillas, estantes y veladores fabricar al mes, con horas de corte (452 h), horas de armado (306 h), litros de barniz (281 L) y un tope de 126 sillas vendibles (`modelos/u07_e04_carpinteria.mod`). La corrida imprime:

```
margen = 5168200
x[mesa] = 40
x[silla] = 126
x[estante] = 20
x[velador] = 0
costo_reducido[mesa] = 0
costo_reducido[silla] = 0
costo_reducido[estante] = 0
costo_reducido[velador] = -3100
holgura[corte] = 0
holgura[armado] = 0
holgura[barniz] = 15
dual[corte] = 7400
dual[armado] = 5300
dual[barniz] = 0
dual_sillas = 1600
```

(a) ¿Qué restricciones están activas? (b) ¿Cuánto vale una hora más de corte y una de armado? (c) Un maestro ofrece 10 h extra de armado a $4.000 la hora: ¿conviene y cuánto se gana? (d) ¿Qué significa el −3100 del velador? (e) ¿Y el 1600 de las sillas?

7.5 · Del modelo semicompacto al compacto (intermedio). Una fábrica de mermeladas tiene este modelo, con una restricción escrita a mano por recurso:

```
{string} Sabores = ...;
float margen[Sabores] = ...;
float uc[Sabores] = ...;   float C = ...;   // cocción (min/lote) y minutos disponibles
float ue[Sabores] = ...;   float E = ...;   // envasado (min/lote)
float ua[Sabores] = ...;   float Z = ...;   // azúcar (kg/lote)
dvar float+ lotes[Sabores];
maximize sum(s in Sabores) margen[s]*lotes[s];
subject to {
  Rt_coccion:  sum(s in Sabores) uc[s]*lotes[s] <= C;
  Rt_envasado: sum(s in Sabores) ue[s]*lotes[s] <= E;
  Rt_azucar:   sum(s in Sabores) ua[s]*lotes[s] <= Z;
}
```

Los datos de la semana:

Por lote	Frutilla	Damasco	Mora	Naranja	Disponible
Margen	$38.000	$30.000	$45.500	$33.500	
Cocción (min)	50	40	60	45	2.350
Envasado (min)	20	20	25	15	925
Azúcar (kg)	12	10	14	11	560

Reescríbelo en forma compacta: un conjunto `Recursos`, un parámetro de dos índices, un solo `forall` con nombre y un `dexpr` con lo usado. Escribe el `.dat`, córrelo e informa lotes, lo usado y los precios sombra. ¿Qué cambia en cada versión si mañana aparece un cuarto recurso (etiquetas)?

7.6 · Empanadas congeladas en cuatro semanas (intermedio). Una fábrica de empanadas congeladas planifica cuatro semanas con demanda de 370, 460, 515 y 290 cajas. Parte con 45 cajas en la cámara de frío. En turno normal produce hasta 410 cajas por semana a $1.750 la caja; en turno extra, hasta 160 más a $2.480. Guardar una caja de una semana a la siguiente cuesta $135. Escribe el modelo con `range Semanas = 1..T;`, el balance de la semana 1 y el de las demás con filtros (`t == 1` y `t > 1`), y minimiza el costo. Informa el plan y los precios sombra de los cuatro balances. ¿Por qué el de la semana 1 es menor que el costo del turno extra?

7.7 · Lavandería que enciende máquinas (intermedio). Una lavandería industrial tiene que lavar 1.340 kg esta semana. Tiene tres lavadoras:

Lavadora	Capacidad (kg/semana)	Costo de encenderla la semana	Costo por kg
A	920	$61.000	$38
B	680	$43.500	$45
C	470	$24.800	$61

(a) Escribe el modelo con `dvar boolean y[Lavadoras]` y la restricción que une los kilos con el encendido. (b) Córrelo: ¿qué máquinas se encienden? ¿Por qué no la B, que lava más barato que la C? (c) Agrega al `execute` una línea que imprima `Pedido.dual`: ¿qué pasa? (d) Cambia `boolean` por `float y[Lavadoras] in 0..1` y vuelve a correr. ¿Qué plan da y por qué no sirve?

7.8 · De la planilla de Solver a OPL (intermedio). Un taller de mochilas tiene armado su modelo en Excel: las celdas B3:D3 son las mochilas urbana, trekking y escolar; F6 calcula el margen con `SUMAPRODUCTO` (márgenes $58.300, $68.000 y $31.700); F8:F9 calculan las horas de costura (3, 4 y 2 por mochila) y de corte (2, 2 y 1); G8:G9 tienen las horas disponibles (342 y 196), y C3 no puede pasar de 52 (lo que se vende de trekking). Solver maximiza F6 cambiando B3:D3, con F8:F9 <= G8:G9 y C3 <= 52, variables no negativas y método Simplex LP. (a) Escribe el `.mod` y el `.dat`. (b) Di qué línea de OPL corresponde a cada elemento de la planilla. (c) Córrelo y llena las columnas del informe de sensibilidad que daría Solver: Valor final y Costo reducido de cada mochila; Valor final y Precio sombra de cada restricción.

7.9 · Corre, pero miente (desafío). Con el `.mod` del caso integrador (sección 13), dos estudiantes corren dos `.dat` distintos. Ninguno da error. Estas son partes de sus salidas:

```
Versión A                          Versión B
margen = 3963250                   margen = 1418.4
x[poleras] = 1092.5                x[poleras] = 0
x[polerones] = 520                 x[polerones] = 0.36
x[bolsas] = 423.75                 x[bolsas] = 0
x[gorros] = 0                      x[gorros] = 0
holgura[planchado] = 0             usado[planchado] = 18
dual[planchado] = 1.5              dual[planchado] = 78.8
costo_reducido[gorros] = -697.5    holgura[tinta] = 29491
```

Sin ver los `.dat`, diagnostica qué está mal en cada uno, en qué línea del `.dat` y qué revisión lo habría detectado. Luego compruébalo con `modelos/u07_e09_error_a.dat` y `u07_e09_error_b.dat`.

7.10 · Distribuidora del Maule (desafío). Una distribuidora reparte cajas desde tres bodegas a cuatro locales. Costo por caja en pesos:

	L1	L2	L3	L4	Oferta (cajas)
Talca	146	212	264	188	535
Linares	231	127	183	249	615
Parral	297	238	114	176	385
Pedido (cajas)	305	265	345	218	

(a) Escribe el modelo con `dvar float+ x[Bodegas][Locales]` y córrelo. (b) ¿Qué bodega es el cuello de botella y cuánto vale una caja más de oferta ahí? Explica el signo. (c) ¿Qué dice el precio sombra del pedido de L4? (d) ¿Cuánto tendría que bajar el costo de Linares a L3 para que esa ruta se use? (e) Un vecino de Parral ofrece guardar 40 cajas más a $10 cada una: ¿conviene? Compruébalo corriendo el modelo con la oferta ampliada.

---

12. Pautas

7.1. Modelo en `modelos/u07_e01_cerveceria.mod`:

```
dvar float+ L;      // barriles de lager al mes
dvar float+ S;      // barriles de stout al mes

dexpr float Maceracion   = 3*L + 5*S;     // horas de maceración usadas
dexpr float Fermentacion = 2*L + 2*S;     // tanques-semana usados

maximize 23000*L + 29000*S;

subject to {
  Rt_maceracion:   Maceracion   <= 150;
  Rt_fermentacion: Fermentacion <= 80;
}
```

Salida: `L = 25`, `S = 15`, margen $1.010.000; se usan las 150 h y los 80 tanques-semana. Precios sombra: $3.000 por hora de maceración y $7.000 por tanque-semana. Los dos recursos aprietan; si hay que ampliar uno, el tanque de fermentación rinde más por unidad.

7.2. (a) Se deciden los kilos de cada ingrediente de un lote de 250 kg. Se busca el menor costo. Limitan: el lote pesa exactamente 250 kg, la proteína y la grasa no bajan de su mínimo y la grasa no pasa de su máximo. (b) Con I ingredientes, N nutrientes, c_i costo, a_{ni} aporte en %, m_n y M_n mínimo y máximo en %, K el lote y k_i los kilos: $\min \sum_i c_i k_i$ s.a. $\sum_i k_i = K$; $\sum_i a_{ni} k_i \geq m_n K \ \forall n$; $\sum_i a_{ni} k_i \leq M_n K \ \forall n : M_n > 0$; $k_i \geq 0$. (c) El filtro crea la restricción `Techo` solo para los nutrientes con máximo positivo: una sola, la de la grasa. (d) Porque ambos lados de la desigualdad quedan multiplicados por 100: $\sum_i 0{,}58\, k_i \geq 0{,}24\, K$ es lo mismo que $\sum_i 58\, k_i \geq 24\, K$. (e) El lote lleva 75 kg de pollo, 165 kg de maíz, 10 kg de aceite y nada de arroz, por $303.600. El arroz tiene costo reducido 369,4: tendría que bajar $369 por kg para entrar. Los precios sombra son 34,80 (proteína) y 17,45 (grasa), en pesos por cada «%·kg» del lado derecho; el de `Total` es 204,78. Subir la proteína mínima un punto sube el lado derecho en $K = 250$, así que el lote cuesta unos $8.699 más. La grasa queda con holgura 1.500 en el techo (está en el piso).

7.3. OPL muestra los errores de a uno: corriges, corres y aparece el siguiente.

Línea con error	Mensaje de OPL	Arreglo
`dvar float cap[Recursos] = ...;`	`syntax error, unexpected ...`	`float cap[Recursos] = ...;` (es un dato)
`dvar float x[Plantas]` sin `;`	`syntax error, unexpected maximize, expecting ';'`	agregar `;`
`dvar float x[Plantas]` sin `+`	ninguno: en este vivero da el mismo plan	`float+`; en 7.10 la falta del `+` vuelve el modelo no acotado
`margen[p] * x[i]`	`Name "i" does not exist.`	`x[p]`
`uso[p][r]`	`Index out of bound for array "uso": lavanda.`	`uso[r][p]`, como se declaró
`x[p] = demanda[p];`	`syntax error, unexpected =`	`x[p] <= demanda[p];` (la demanda es un máximo, no una meta)

El último es doble: `==` compila, pero obligaría a producir toda la demanda. El modelo corregido (`modelos/u07_e03_vivero.mod`) da 230 macetas de lavanda, 410 de romero (su tope) y 60 de suculentas, con margen $674.150. Los dos recursos aprietan: un minuto de trasplante vale $215 y un litro de sustrato $70; una maceta más de romero vendible vale $60.

7.4. (a) Corte y armado (holgura 0) y el tope de sillas; sobran 15 L de barniz. El plan (40 mesas, 126 sillas, 20 estantes) deja $5.168.200. (b) $7.400 la hora de corte y $5.300 la de armado. (c) Sí: 10 horas valen $53.000 y cuestan $40.000, se ganan $13.000. El verificador re-resuelve con 306 + 10 h y el margen sube exactamente $53.000, porque 10 h están dentro del rango en que el precio sombra no cambia (unidad 12). (d) Cada velador que se obligue a fabricar baja el margen en $3.100: sus horas de corte y armado valen más que su margen. Su margen tendría que subir $3.100 para que entre. (e) Si se pudieran vender 127 sillas en vez de 126, el margen subiría $1.600.

7.5. Versión compacta (`modelos/u07_e05_mermeladas.mod`):

```
{string} Sabores  = ...;
{string} Recursos = ...;

float margen[Sabores]           = ...;   // CLP por lote
float uso[Recursos][Sabores]    = ...;   // minutos o kg por lote
float disp[Recursos]            = ...;   // minutos o kg por semana

dvar float+ lotes[Sabores];

dexpr float usado[r in Recursos] = sum(s in Sabores) uso[r][s] * lotes[s];

maximize sum(s in Sabores) margen[s] * lotes[s];

subject to {
  forall(r in Recursos)
    Disponible:
      usado[r] <= disp[r];
}
```

Plan: 20 de frutilla, 15 de mora y 10 de naranja (nada de damasco), con margen $1.777.500. Se usan los 2.350 minutos de cocción, los 925 de envasado y los 560 kg de azúcar. Precios sombra: $400 por minuto de cocción, $300 por minuto de envasado y $1.000 por kg de azúcar. El damasco tiene costo reducido −2.000: le faltan $2.000 de margen por lote. Con un cuarto recurso, la versión semicompacta pide un parámetro nuevo, un escalar nuevo y una restricción nueva en el `.mod`; la compacta solo pide una fila más en el `.dat`.

7.6. El balance en OPL (`modelos/u07_e06_empanadas.mod`):

```
  forall(t in Semanas: t == 1)
    Balance1:
      inv0 + r[t] + e[t] - I[t] == demanda[t];
  forall(t in Semanas: t > 1)
    Balance:
      I[t-1] + r[t] + e[t] - I[t] == demanda[t];
```

Semana	1	2	3	4
Turno normal	410	410	410	290
Turno extra	0	0	70	0
Inventario al cierre	85	35	0	0
Precio sombra del balance	$2.210	$2.345	$2.480	$1.750

Costo total: $2.849.800. Se adelanta producción normal en las semanas 1 y 2 (guardar cuesta $135 por semana, mucho menos que la diferencia de $730 con el turno extra) y en la semana 3 igual faltan 70 cajas en turno extra. El precio sombra es lo que cuesta una caja más de demanda en esa semana. En la semana 1 vale $2.210 y no $2.480 porque esa caja sale del inventario que iba a las semanas 2 y 3; lo que falta al final se hace en turno extra en la semana 3, y se ahorran dos semanas de guardarla: $\mathbf{2.480 - 2 \cdot 135 = 2.210}$.

7.7. El encendido se modela con `kg[l] <= cap[l] * y[l]`: si `y` es 0 la máquina no lava; si es 1 lava hasta su capacidad (`modelos/u07_e07_lavanderia.mod`). (b) Se encienden A y C: A lava 920 kg y C 420 kg, con costo $146.380. A sola no alcanza, y A con B cuesta más: B tiene un costo fijo de $43.500 contra $24.800 de C, y esa diferencia pesa más que los $16 por kg que ahorra en 420 kg. (c) Imprime `undefined` y CPLEX avisa `Not available for mixed-integer problems`: en un modelo entero no hay precios sombra. (d) Con `y` continua, el costo fijo se reparte por kilo: A cuesta $104,3 por kg lleno, B $109,0 y C $113,8. El modelo llena A y usa B a medias (y_B = 0,62, 420 kg), con costo $141.728. No sirve: no se puede encender el 62 % de una lavadora y pagar el 62 % del costo fijo. Esa relajación es una cota inferior del costo real, no un plan.

7.8. (a) `modelos/u07_e08_mochilas.mod` y `.dat`. (b) B3:D3 son `dvar float+ x[Modelos]`; F6 es `dexpr float Margen`; F8:F9 son `dexpr float usado[p in Procesos]`; F8:F9 <= G8:G9 es el `forall(p in Procesos) Horas:`; C3 <= 52 es la restricción `Trekking:`; «no negativas» es el `+`. (c) El informe, con los números de OPL:

Variable	Valor final	Costo reducido
Urbana	50	0
Trekking	48	0
Escolar	0	−2.300

Restricción	Valor final	Precio sombra	Lado derecho
Costura	342	9.700	342
Corte	196	14.600	196
Trekking	48	0	52

Margen $6.179.000. Una hora más de costura vale $9.700 y una de corte $14.600; quedan 4 mochilas de trekking por vender. A la escolar le faltan $2.300 de margen para entrar.

7.9. Versión A: falta un margen. Los gorros, que en el caso salen en su tope, salen en 0 con costo reducido −697,5. Ese número es exactamente lo que «cobran» sus recursos: 15 s de planchado a 1,5 más 6 g de tinta a 112,5. O sea, el modelo cree que el gorro no deja margen: el arreglo `margen` del `.dat` tiene cuatro números para cinco prendas y OPL completó con 0. Revisión que lo detecta: contar los números de cada arreglo contra su conjunto. Con el margen bien escrito vuelve el plan de la sección 13; con el error, el

margen cae a 3.963.250 y el plan cambia (1.092,5 poleras, 423,75 bolsas). **Versión B: unidades mezcladas.** El taller produce 0,36 polerones y gana $1.418,4: se usan «18» de planchado y sobran 29.491 g de tinta. La capacidad se escribió en horas (40 y 18) mientras el consumo está en segundos por unidad; el precio sombra 78,8 por «segundo» es absurdo frente al 1,5 del caso. Revisión: escribir la unidad de cada número en un comentario del `.dat` y mirar si lo usado se parece a lo disponible.

7.10. Modelo en `modelos/u07_e10_transporte.mod`. (a) Talca envía 305 a L1 y 178 a L4; Linares, 265 a L2; Parral, 345 a L3 y 40 a L4. Costo: $158.019. (b) Parral es la única bodega sin holgura (a Talca le sobran 52 cajas y a Linares 350); su precio sombra es −12: una caja más de oferta en Parral *baja* el costo en $12, porque reemplaza una caja que Talca manda a L4 a $188 por una de Parral a $176. En un mínimo, una capacidad que aprieta tiene dual negativo. (c) Atender una caja más en L4 cuesta $188 (sale de Talca, que tiene holgura). (d) Linares a L3 tiene costo reducido $57: su costo tendría que bajar más de $57 por caja (de 183 a menos de 126) para usarse. (e) Ampliar Parral en 40 cajas ahorra $480 y cuesta $400: conviene, por $80 a la semana. La corrida con oferta 425 confirma el ahorro, porque las 40 cajas siguen desplazando envíos de Talca a L4.

---

13. Caso integrador · Estampa Sur corre su mezcla en OPL

> Este caso sigue a Estampa Sur por todo el ramo. La unidad 4 planteó la mezcla de las cinco prendas de una semana; acá ese mismo modelo se separa en `.mod` y `.dat` y se corre en CPLEX. Si no has hecho la unidad 4, los datos están en el `.dat`.

El modelo (`modelos/u07_mezcla.mod`). Las restricciones de demanda se escriben como restricciones con nombre, y no como cotas de la variable, para poder pedirles su precio sombra:

```
/*********************************************
 * Unidad 7 · Implementación en OPL
 * Estampa Sur: mezcla semanal de cinco prendas (el modelo de la unidad 4)
 * Datos en u07_mezcla.dat. Base abierta de ingeniería.
 *********************************************/

// CONJUNTOS
{string} Prendas  = ...;          // lo que se puede producir
{string} Recursos = ...;          // estampado, planchado y tinta

// PARÁMETROS
float margen[Prendas]           = ...;   // CLP por unidad
float uso[Recursos][Prendas]    = ...;   // segundos o gramos por unidad
float cap[Recursos]             = ...;   // segundos o gramos por semana
float demanda[Prendas]          = ...;   // unidades por semana

// VARIABLES DE DECISIÓN
dvar float+ x[Prendas];                  // unidades a producir en la semana

// EXPRESIONES (se calculan, no se deciden)
dexpr float Margen = sum(i in Prendas) margen[i] * x[i];
dexpr float usado[r in Recursos] = sum(i in Prendas) uso[r][i] * x[i];

// OBJETIVO
maximize Margen;

// RESTRICCIONES
subject to {
  forall(r in Recursos)
    Capacidad:
      usado[r] <= cap[r];

  forall(i in Prendas)
    Demanda:
      x[i] <= demanda[i];
}

// SALIDA: una línea «clave = valor» por resultado
execute {
  writeln("margen = ", Margen);
  for (var i in Prendas)
    writeln("x[", i, "] = ", x[i]);
  for (var r in Recursos) {
    writeln("usado[", r, "] = ", usado[r]);
    writeln("holgura[", r, "] = ", Capacidad[r].slack);
    writeln("dual[", r, "] = ", Capacidad[r].dual);
  }
  for (var i in Prendas) {
    writeln("dual_demanda[", i, "] = ", Demanda[i].dual);
    writeln("costo_reducido[", i, "] = ", x[i].reducedCost);
  }
}
```

Los datos (`modelos/u07_mezcla.dat`), en segundos y gramos para que todo sume en la misma unidad:

```
/*********************************************
 * Unidad 7 · datos de la mezcla de Estampa Sur (una semana)
 * Mismas cifras que el caso integrador (unidad 4).
 *********************************************/

Prendas  = {"poleras", "polerones", "doble", "bolsas", "gorros"};
Recursos = {"estampado", "planchado", "tinta"};

margen  = [1380 3940 2270 960 1170];

uso = [
  [30 90 110 20 30]      // estampado (s/unidad)
  [20 50  30 40 15]      // planchado (s/unidad)
  [12 25  22  8  6]      // tinta (g/unidad)
];

cap     = [144000 64800 29500];        // 40 h, 18 h (en segundos) y 29.500 g
demanda = [1450 520 380 860 690];
```

La corrida. `python herramientas/opl.py ramos/optimizacion/modelos/u07_mezcla.mod ramos/optimizacion/modelos/u07_mezcla.dat` da `OBJECTIVE: 4289275` y:

```
margen = 4289275
x[poleras] = 833.75
x[polerones] = 520
x[doble] = 0
x[bolsas] = 294.375
x[gorros] = 690
usado[estampado] = 98400
holgura[estampado] = 45600
dual[estampado] = 0
usado[planchado] = 64800
holgura[planchado] = 0
dual[planchado] = 1.5
usado[tinta] = 29500
holgura[tinta] = 0
dual[tinta] = 112.5
dual_demanda[poleras] = 0
costo_reducido[poleras] = 0
dual_demanda[polerones] = 1052.5
costo_reducido[polerones] = 0
dual_demanda[doble] = 0
costo_reducido[doble] = -250
dual_demanda[bolsas] = 0
costo_reducido[bolsas] = 0
dual_demanda[gorros] = 472.5
costo_reducido[gorros] = 0
```

Lectura.

Pregunta	Respuesta de la salida
¿Qué se produce?	833,75 poleras básicas, 520 polerones (tope), 294,375 bolsas y 690 gorros (tope); nada de estampado doble
¿Cuánto deja?	$4.289.275 de margen en la semana
¿Qué recurso sobra?	estampado: se usan 98.400 de 144.000 segundos; sobran 45.600 s (12,7 h), dual 0
¿Qué aprieta?	planchado y tinta, con holgura 0
¿Cuánto vale un segundo más de planchado?	1,5 pesos: $5.400 por hora
¿Y un gramo más de tinta?	112,5 pesos
¿Conviene vender más polerones o gorros?	sí: una unidad más de demanda vale 1.052,5 (polerones) y 472,5 (gorros)
¿Por qué no se hace estampado doble?	costo reducido −250: cada una baja el margen en $250

La cadena. El verificador resuelve el mismo modelo con scipy y exige que objetivo, cantidades, holguras y precios sombra coincidan con OPL y con el script del caso (`CADENA: OK`). Es la misma mezcla de la unidad 4, ahora lista para correr con cualquier semana de datos.

Lo que pasa a las unidades siguientes. La unidad 8 toma la versión de dos productos (poleras y polerones) y la resuelve a mano con el método simplex, para ver de dónde salen estos precios sombra. La unidad 12 usará los $5.400 por hora de planchado contra los $6.750 que cuesta la hora extra.

---

14. Para seguir

En los libros del ramo.

- Winston, *Investigación de operaciones: aplicaciones y algoritmos* (4.ª ed., 2005): los capítulos de formulación de modelos lineales traen muchos problemas para practicar la traducción a OPL.
- Hillier y Lieberman, *Introducción a la investigación de operaciones* (7.ª ed., 2001): su capítulo de programación lineal discute la formulación en planilla, útil para la sección 8.

Abiertos y gratis.

- IBM, *OPL Language Reference Manual* (CPLEX Optimization Studio 22.1.1), en inglés: sintaxis completa de conjuntos, parámetros, restricciones, conexión con planillas y el lenguaje del `execute`. https://www.ibm.com/docs/en/icos/22.1.1?topic=opl-language-reference-manual

Con el tutor. La carpeta `tutor/` trae el contexto de este ramo. Pégalo al inicio de la conversación con cualquier IA y pídele, por ejemplo: «Dame un `.mod` de OPL con tres errores escondidos y corrígeme cuando los encuentre».
